

Pokémon AI Team Building & Statistics

Project Report

Christian Biermann

Project Idea

The idea for the project is to have Artificial Intelligence, specifically a Large Language Model Chatbot to be able to generate Pokémon team that is:

1. Valid given a specific tournament regulation
2. Considered strong within the format to where it can be used in actual tournament

I chose this idea because I personally play competitive Pokémon, and one challenge I have faced, as well as many other players I have seen online and spoken to, is team building. Team building is an important part of competitive Pokémon, but it can also be difficult to understand because there are many factors involved.

One major factor is team synergy, meaning whether the Pokémon on a team work well together. Another important factor is each Pokémon's move set. For example, a more physically oriented Pokémon should usually have more physical attacks than special ones, while a support Pokémon may focus less on attacking moves and more on moves that help create advantages in battle, such as inflicting paralysis to slow opposing Pokémon down and allow its team to move first.

Methodologies

There are a few methodologies that I used in the project that was each used for a specific purpose.

Ollama AI Chatbot

The first one to discuss is the chat bot itself. For this project, I decided to go with the open source Ollama AI Chatbot specifically version Qwen3:14b. I was not sure about which model to get but looking online and seeing the specifications of my computer, this specific model was the best I could do. With a better computer, the model could be newer, which could mean a better output by chance. This model would be given the some input data such as the Pokémon that is allowed within the specific regulation, the rules of the regulation, the valid items allowed in the

regulation, and finally a schema for the AI to output the team in a specific way to be used later in a battle simulation.

Monte Carlo Simulation (Pokémon Showdown API)

The second piece of technology is creating a Monte Carlo Simulation by using the showdown battle simulation that is found on the Pokémon Showdown website. This website is where people can build teams within specific regulations and battle against other players and gain points to represent how good the player is. This site also has a battle simulation API that can be used to mimic how a battle could go between two teams. This API is what will be used to get the wins and losses between the Ollama AI team against actual tournament teams. These tournament teams will be gotten from a website containing teams used in an actual tournament based on the regulation.

Genetic Algorithm

The third piece of technology is creating a Genetic Algorithm (GA) to improve the Ollama AI team. Based on a valid input, have the input go through multiple iterations of changes, view each to change to see how good the change was, and keep the better solutions. In this case, the valid input is a valid team based on a regulation, and the multiple iterations are constantly using the battle simulation after making a change to team while keeping the overall team valid, and finally, keeping the final best team after the GA is done.

To be more specific, after a simulation, the team goes through the GA with a chance to change certain parts of the team such as the Pokémon, moves, nature, or held item. Once this change occurs, the team gets put through the simulation to determine if this team or the previous is better and keep the better one. This constantly happens throughout the GA, specifically after each full iteration. To explain how the GA works in the project, there are 10 teams per iteration and there are 12 iterations total that the GA will process. I also run the GA repeatedly 3 times to improve the optimization.

Logistic Regression and LIME

The final piece of technology used in the project is a Logistic Regression Model and a LIME explanation to clarify if the Ollama AI team is a strong team in the regulation. For the Logistic Regression Model, the features used to help describe the team can be a lot, so I went with the most important ones and the most seen ones. For example, important ones are counting the physical, special, and support Pokémon on a team, while a most seen one could be how many Pokémon has the move fake out. The full list of features will be shown in the Implementation section.

The LIME explanation is used to give a visual on the features of a specific team. This specific team used for the Logistic Regression and LIME will be the best team output from the Genetic Algorithm as the input for the model.

Implementation

The implementation process will be a step-by-step explanation on how each methodology is connected to each other and what the input and output is for each.

The first thing I want to note is that the uses of each of these methodologies is used within the python notebook, but any function calls such as checking the validity of the AI generated team or calling the Genetic Algorithm comes from normal python scripts. The Monte Carlo Simulation

So, starting off with the Ollama AI is used first. The input for it specifically is the Pokémon that is valid for Pokémon Tournament Regulation I that started this month, the rules of regulation I, the valid items in the regulation, and a schema that makes the AI output the team in a specific style. The last piece of input that can be changed is the prompt for the AI that comes from the user determining what kind of team to make. This can be a weather team such as a rain or sun team, or it can be a team based around a specific Pokémon. For the current state of the project, I gave the prompt of outputting a rain team.



The figure above is the output of the Ollama AI after asking for a rain team inputted into a website called pokepaste to show a visual of the team. This will be the starting team that will be used in the simulation and the start of the Genetic Algorithm.

The generated team is then checked to determine whether it is valid under the selected regulation. If the team is not valid, it is passed back through the Ollama AI, which by that stage produces a valid team. Once a valid team is obtained, it is saved as a JSON file so it can be used by the other methodologies, specifically the Monte Carlo simulation through the Showdown API. The Showdown API takes the AI-generated team along with tournament teams stored in JSON files and simulates a set number of battles against each one. Based on how many of those battles are won, the system produces a score for the team.

Once the score is obtained after the first simulation, the team is then sent through the Genetic Algorithm to try and improve the team. Once the whole iteration is done, the best team from that iteration is then saved to be ran through the simulation again. This is then repeated until the Genetic Algorithm finishes.

Once the GA is completed, the information on the best scoring team overall is saved and that team will be used in the Logistic Regression and LIME.

For the Logistic Regression, a specific set of features were manually created to determine if the team was a strong team or not. As said before some were considered default features and some were personally done because they are strong contenders to consider when determining if the team is strong or not. The features used are:

- Restricted_Count
- Physical_Count
- Special_Count
- Support_Count
- Priority_User_Count
- Protect_Count
- Fake_Out_Count
- Redirection_Count
- Pivot_Move_Count
- Setup_Move_Count
- Spread_Move_User_Count
- Status_Move_Count
- Weather_Setter_Count
- Terrain_Setter_Count
- Intimidate_Count
- Has_Fake_Out
- Has_Tailwind
- Has_Trick_Room
- Has_Redirection
- Has_Wide_Guard
- Has_Speed_Control

These were the features I chose to consider when determining a strong team, although there can be many other variables to consider and include. The model would look at each team and view these features and depending on the count and

Boolean variables that were returned with these features, a team can be determined if it is a strong team for the regulation.

I chose to split the data with 20% testing data and 80% training data. Also, the data that the Logistic Regression was using was the combination of the Ollama AI teams and the ChatGPT teams to help make its predictions. I also gave it a label to determine if the team was strong based on a threshold number, in this case the number is 50. If the score of the team is above 50, it's a strong team, if below, it is a weak team.

What was specifically outputted from the Logistic Regression was an accuracy score, intercept score, a classification report, and a confusion matrix.

Below are the numbered statistics:

Accuracy Score: 77.78%

Intercept Score: -12.02%

Classification Report:

Label 0:

- Precision: 0.99
- Recall: 0.76
- F1 Score: 0.86

Label 1:

- Precision: 0.28
- Recall: 0.92
- F1 Score: 0.43

Confusion Matrix:

```
[100 31]
[  1 12]
```

The overall model was accurate in guessing a strong team 77% of the time which okay, but there is still some room for improvement. Looking at the numbers for each label, the model was pretty accurate in describing a weak team but not that

correct when labelling a strong tournament team. This is most likely due to lack of label 1 data considering the Ollama AI specific team data was all 0 indicating the AI generated such a weak team that even running the team through the Genetic Algorithm, it could not be improved enough to meet the threshold. This could be solved through either improving the Algorithm all together or increasing the number of total teams, iterations, and runs for the team to go through.

When it came to the LIME explanations, I chose to do those separately as in each best team for each AI was used in the LIME explanation. Below are a visual explanation for each AI on the final best team from the Genetic Algorithm.

Ollama AI



ChatGPT AI



Viewing the two explanations there are some obvious points that can be seen as to why the Ollama AI scored so poorly, such as the ChatGPT having 2 restricted which in this case means either a legendary or mythical Pokémon and these specific type of Pokémon have an extremely high base stat total and strong moves and the Ollama AI having none. Another clear cause is how many status moves the Ollama AI team has compared to ChatGPT. More status moves means less opportunities for attacking moves which means they will do almost no damage.

Conclusion

Overall, I would say the model has a decent start but there are many improvements that would need to be made to reach the baseline that ChatGPT made. A few obvious improvements would be towards the Genetic Algorithm, improving what gets changed or how often it gets changed. Another thing is possibly creating our own Battle Simulation as the Monte Carlo Simulation. This is because I am unsure how good the Pokémon Showdown Battle Simulation API is and if we make our own, we can edit it to make improvements. As typical with all AI, there are many ways to improve it but these two are a pretty good start to help the Ollama AI produce a team that meets the threshold.